

SWEET1N

QR-Powered Hotel & Restaurant Management System — Full-Stack Project Documentation

1. Overview

Sweet1n is a QR-code powered hotel and restaurant ordering and management platform. A customer scans a QR code on their table, which opens a digital menu — no app install or account required. The platform manages the full order lifecycle (ordering, payment, kitchen preparation, service), gives staff role-specific dashboards, offers AI-assisted menu suggestions for guests with dietary needs, and gives management real-time sales analytics.

2. Objectives

- Replace physical menus and manual order-taking with a contactless, QR-based digital ordering flow.
- Give customers real-time visibility into their order status from placement to serving.
- Support multiple payment methods, including automated mobile money verification.
- Help guests with allergies, health conditions, or dietary preferences choose suitable dishes via AI-assisted suggestions.
- Give management real-time sales analytics to guide menu, staffing, and pricing decisions.
- Coordinate cashier, kitchen, and waiter teams through role-specific dashboards fed by one shared order pipeline.

3. System Architecture

The system is composed of four layers that communicate through Firestore's real-time listeners and Cloud Functions:

- Client layer — Next.js app serving the public QR menu, order tracking page, and all staff dashboards (responsive, mobile-first).
- Application/logic layer — Firebase Cloud Functions handling order transitions, payment webhooks, AI recommendation requests, and scheduled analytics aggregation.
- Data layer — Cloud Firestore for live data (tables, orders, menu, staff, transactions) and Firebase Storage for images.
- External services — STRIP API (and optional card processor) for payments; an LLM API for generating dietary-suggestion explanations.

4. Technology Stack

Layer	Choice & Role
Frontend	Next.js (React) — SSR for fast menu loads, file-based routing for public + role dashboards, PWA-friendly.
Backend logic	Firebase Cloud Functions — order processing, payment webhooks, AI recommendation calls, analytics aggregation.
Database	Cloud Firestore — real-time NoSQL store for orders, menu, tables, staff, transactions.
Authentication	Firebase Authentication — staff/role-based login (email or phone); customers remain unauthenticated.
File storage	Firebase Storage — menu item images, staff profile photos.
Payments	M-Pesa Daraja API (STK Push) as primary; Stripe/Flutterwave as optional card fallback; cash handled manually by cashier.
AI/Recommendations	Cloud Function endpoint combining rule-based dietary filtering with an LLM-generated explanation (e.g., Claude API) for suggested dishes.
Analytics & charts	Scheduled Cloud Functions aggregate daily sales into Firestore; Recharts (or similar) renders charts in the manager dashboard.

Hosting	Firebase Hosting / Google cloud/Vercel for the Next.js frontend; Firebase project for backend services.
---------	---

5. User Roles & Dashboards

The system supports six user types, each with a purpose-built interface:

Role	Responsibilities
Customer	Scans table QR code, browses AI-assisted digital menu, sets dietary preferences, places order, chooses payment method, tracks order status live, no login required.
Cashier	Views incoming orders, confirms cash payments manually, reviews auto-confirmed digital payments, releases orders to the kitchen queue.
Kitchen Staff	Views paid orders on a kitchen display, updates status (Preparing → Ready), flags item unavailability.
Waiter	Sees "Ready" orders per table, marks orders as served, manages table service and reassignment.
Manager	Full oversight: tables/QR codes, staff accounts, menu & dietary tags, sales analytics, payment reconciliation, reports.
Receptionist	Manages table availability/reservations, walk-in assignment, guest check-in assistance.

6. Order & Payment Workflow

1. Customer scans the QR code on their table → digital menu loads (table identity is embedded in the QR link).
2. Customer optionally sets a dietary preference (e.g., diabetic, nut allergy, vegan) → AI-assisted suggestions highlight suitable dishes.
3. Customer adds items to cart and places the order, selecting a payment method (cash, M-Pesa, or card).
4. For digital payments, an STK Push (or card checkout) is triggered; a provider webhook confirms payment automatically. For cash, the order waits for cashier confirmation.
5. Once payment is confirmed, the order moves to the Kitchen dashboard as a preparation ticket.
6. Kitchen staff update status as it is prepared (Preparing → Ready).
7. Waiters see "Ready" orders per table and mark them Served.
8. The customer's order-tracking page updates live at every stage: Placed → Paid → Preparing → Ready → Served.
9. Completed orders feed into the analytics engine for daily/weekly reporting.

7. Payment Integration

Sweet1n supports three payment paths, all reconciled into a single transactions log for accounting:

Method	Flow
Cash	Cashier verifies payment received in person and manually marks the order "Paid" in the dashboard.
Mobile Money (M-Pesa)	Customer selects STRIPE options at checkout → backend triggers an STK Push to the customer's phone → customer enters Credential PIN → STRIPE sends a callback to a Cloud Function → order is auto-marked "Paid" and pushed to the kitchen queue.
Card (optional)	Card payment via Stripe/Flutterwave checkout session for international or card-preferring guests; webhook confirms payment the same way as M-Pesa.

Note: payment provider webhooks are signature-verified before an order is marked "Paid," and every transaction is logged independently of the order record for audit and reconciliation purposes.

8. AI-Assisted Dietary Menu Suggestions

Every menu item carries structured tags: allergens (nuts, dairy, gluten, shellfish, etc.), dietary category (vegan, vegetarian, diabetic-friendly, low-sodium, keto), spice level, and calorie estimate.

How it works

- The customer optionally selects a dietary need or preference when opening the menu — no account needed, just a lightweight on-page selector.
- A rule-based filter first removes any item containing a flagged allergen or conflicting with the stated preference.
- From the remaining items, an AI layer (an LLM call from a Cloud Function) ranks and annotates suggestions, e.g., explaining in plain language why a grilled fish dish suits a diabetic-friendly request.
- Suggestions are clearly labelled as recommendations, with a visible disclaimer that they are not medical advice and guests with serious conditions should confirm ingredients with staff.

9. Sales Analytics & Reporting

The manager dashboard includes a real-time analytics module, built from scheduled Cloud Functions that aggregate order and transaction data:

Metric	Description
Revenue tracking	Daily, weekly, and monthly totals; trend comparisons against previous periods.
Best/least sellers	Ranked menu items by quantity sold and revenue contributed.
Peak hours/days	Order-volume heatmap to guide staffing decisions.
Payment method split	Cash vs. M-Pesa vs. card share of total revenue.
Staff performance	Orders served per waiter, average kitchen prep time per item/staff.
Table turnover	Average time from seating to order completion, per table.
Exports	CSV/PDF export of any report range for offline record-keeping.

10. Data Model (Firestore Collections)

Collection	Key Fields
tables	id, qrCodeUrl, status (free/occupied), capacity, currentOrderId
menuItems	id, name, price, category, imageUrl, allergens[], dietaryTags[] (vegan, diabetic-friendly, gluten-free...), available
orders	id, tableId, items[], status (placed/paid/preparing/ready/served), paymentMethod, paymentStatus, createdAt
transactions	id, orderId, method, amount, providerRef (M-Pesa/Stripe id), status, timestamp
staff	id, name, role, contact, authUid, active
analyticsDaily	date, totalRevenue, ordersCount, topItems[], revenueByMethod{}, peakHour
feedback	id, orderId, rating, comment, createdAt

11. Security & Access Control

- Staff authenticate via Firebase Authentication; Firestore security rules restrict each role to only the data and actions it needs (e.g., waiters cannot edit the menu).
- Customers remain unauthenticated but are scoped to their table's order via a session token embedded in the QR link, preventing cross-table interference.

- Payment webhooks are signature-verified; sensitive payment details are never stored client-side.
- All traffic is served over HTTPS; Firestore rules and Cloud Functions enforce role checks server-side, not just in the UI.

12. Non-Functional Requirements

- Performance: SSR menu pages and Firestore real-time listeners keep updates near-instant across all dashboards.
- Scalability: Firebase services scale automatically with order volume; Cloud Functions are stateless.
- Reliability: graceful offline messaging if connectivity drops; failed payment callbacks are retried.
- Usability: mobile-first design for the customer-facing menu, since it is accessed almost exclusively via phone.

13. Future Roadmap

- Loyalty/rewards program for returning guests.
- Multi-branch support for hotel chains.
- Post-order feedback and ratings.
- Push/SMS notifications when an order is ready.
- Multi-language menu support.